https: / / github.com / fhlsteven / CLR_via_CSharp / blob / main / chapters / ch18_CustomAttributes.md 我现在在学习《CLR via C#》这本书的第18章，文章内容如链接所示，我们将

用户:

https://github.com/fhlsteven/CLR_via_CSharp/blob/main/chapters/ch18_CustomAttributes.md

我现在在学习《CLR via C#》这本书的第18章，文章内容如链接所示，我们将围绕此进行讨论，我也会向你请教。

元宝:

好嘞，《CLR via C#》第18章「定制特性（Custom Attributes）」我已经顺着你给的链接看过了，Jeffrey Richter 这一章讲得挺系统——从**用特性 → 定义自己的 Attribute 类 → AttributeUsage 约束 → 构造器/字段允许的数据类型 → 运行期检测（IsDefined / GetCustomAttribute） → 不实例化对象的安全反射（CustomAttributeData） → 条件特性类（Conditional）**，一条链路走完。

这一章有几个地方是读者经常卡住的，你可以看看是不是你接下来想聊的方向，或者你直接抛问题也行：

- **Attribute 的本质**：它不是一个"编译器魔法"，而是**编译期把构造器参数+命名参数序列化进元数据**，运行期通过反射反序列化出来的普通类实例——这个模型搞清了，后面全通。
- `AttributeUsage` **三个属性的坑**：
  - `ValidOn` 用 `AttributeTargets` 位标志控应用范围
  - `AllowMultiple` 默认 false，多次应用同一特性会 CS0579
  - `Inherited` 只对类/方法/属性/事件等有意义，而且**不会在派生类型元数据里多生成记录**，是反射时往上追查的
- **检测时的性能取舍**：`IsDefined` 不构造对象最快；`GetCustomAttribute(s)` 才会真的 new 实例、调构造器、设字段——这也是为什么后面引出 `CustomAttributeData`（防构造器里藏恶意代码，沙箱场景）。
- `ConditionalAttribute` **作用于特性类本身** vs `#if` 的区别：前者是**编译器决定是否把特性元数据写进模块**，特性类本身还在程序集里；后者是整个代码块都不参与编译。

你现在是刚读完想梳理，还是已经有具体哪段代码/哪个概念没太吃透？比如 `Match` 重写那段、`CustomAttributeData` 的安全场景、或者 `Inherited` 那个例子里 `DerivedType` 为啥不继承 `Serializable` 这种，都可以直接扔过来。

用户:

“18.1 使用定制特性”这一节解释一下，同时为什么Attribute后缀可以省略？

元宝:

好，18.1「使用定制特性」是这一章的敲门砖——Richter 没一上来就让你定义自己的 Attribute，而是先把**“怎么用、长什么样、编译后去哪了”**给你铺清楚，免得后面定义完了还不知道这东西到底干嘛用的。

18.1 这一节在讲什么

核心一句话：**定制特性就是“给代码元素贴元数据标签”，编译器把这些标签序列化进 PE 文件的元数据里，运行期靠反射读出来驱动行为。**这一节沿着这条线铺了几件事：

1）FCL 里现成的特性先认一遍
Richter 举了几个你肯定眼熟的例子，让你感知“特性不是魔法，是 CLR/编译器/库已经大量在用”的东西：

- `[DllImport("Kernel32")]` → 告诉 CLR 方法体在非托管 DLL 里
- `[Serializable]` → 告诉序列化格式化器“这个类型的字段可以序列化”
- `[AssemblyVersion("1.0.0")]` → 钉程序集版本号
- `[Flags]` → 枚举变成位标志集合

2）特性能贴在哪
C# 允许几乎任何元数据可表示的东西：`assembly` / `module` / `type` / `field` / `method` / `param` / `return` / `property` / `event`，还可以用前缀显式指定目标，比如：

```
[assembly: SomeAttr]      // 贴到整个程序集
[return: MarshalAs(...)]  // 贴到方法返回值
[param: In, Out]          // 贴到参数
```

多数情况编译器能推断，但有些模糊场合（比如泛型参数、返回值）得手写前缀。

3）特性的本质：Attribute 派生类的实例
符合 CLS 规范要求 → 必须直接或间接从 `System.Attribute` 派生，C# 也只认这种。

应用特性的语法**看起来像调用构造器**，其实就是在描述“将来反射时要怎么 new 这个实例”：

```
[DllImport("Kernel32", CharSet = CharSet.Auto, SetLastError = true)]
```

- `"Kernel32"` → 构造器实参，叫**定位参数（positional）**，必填
- `CharSet = ...` → 设公共字段/属性，叫**命名参数（named）**，可选

4）几种等价写法（这一节的小彩蛋）

```
[Serializable][Flags]
[Serializable, Flags]
[FlagsAttribute, SerializableAttribute]
[FlagsAttribute()][Serializable()]
```

这四种完全一样。推导：每个特性可单开方括号，也可逗号塞同一对；构造器无参时圆括号可省；`Attribute` 后缀可省。

为什么 `Attribute` 后缀可以省

这是 **C# 编译器层的语法糖，不是 CLR 层的。** `System.Attribute` 派生类的命名惯例是 `XxxAttribute`，但应用的时候写 `[Xxx]` 还是 `[XxxAttribute]` 都行——规范 17.2 原文：

> 按照约定，属性类的命名带有 `Attribute` 后缀。 `attribute_name` 可以包含或省略此后缀。

编译器解析 `[Xxx]` 时的实际动作是：**同时去查 `Xxx` 和 `XxxAttribute` 两个类型**，找到哪个用哪个。

⚠️ 但这里有个坑：歧义编译错误

如果**两个都存在**——既有类 `Xxx` 又有类 `XxxAttribute`——编译器不给你"二选一"，直接 CS0104 歧义报错。这时候用逐字标识符 `@` 化解：

```
[@Foo]       // 只匹配 Foo，不匹配 FooAttribute
[@FooAttribute] // 匹配 FooAttribute（逐字不影响后缀查找）
```

> 顺便说：`[Serializable]` 真实类名其实是 `SerializableAttribute`，`[DllImport]` 真实类名是 `DllImportAttribute`——你在元数据里看到的、MSDN 里点进去的，都是带 `Attribute` 后缀的那个；源码里写短的那个，是 C# 编译器帮你补全的。

设计动机就是 Richter 说的"减少击键、提升可读性"——方括号里本来就已经是特性语境了，再写一遍 `Attribute` 纯属视觉噪音。

18.1 读下来其实就建立一个心智模型：**特性 = `Attribute` 派生类实例 + 编译期序列化进元数据 + 运行期反射反序列化。**后面 18.2 开始让你自己写 `class FooAttribute : Attribute`，18.4 才讲 `IsDefined` / `GetCustomAttribute` 怎么把元数据里的字节真的变回对象——这条链路顺下来这一章就通了。

你接下来是往 18.2 走，还是 18.1 里还有哪段没太嚼透？比如那个"定位参数 vs 命名参数"的语义差别，或者 `assembly:` / `module:` 前缀那种不常用场景，都可以继续拆。

▾ 引用 11 篇资料作为参考

1. 《CLR via C#》笔记:第3部分 基本类型(3)-腾讯云开发者社区-腾讯云
2. 属性 - C# language specification
3. 笔记 第18章 定制特性
4. 使用[SomeAttribute]和[Some]修饰类的区别
5. NET CLR via c# 第4版笔记 第18章 定制特性
6. What's with using C# attributes with the 'Attribute' suffix?
7. 《CLR via C#》基本类型.定制特性
8. 为什么我可以使用"[Authorize]"这个缩写而不是实际类名"[AuthorizeAttribute]"?
9. CLR via C# - Chapter 18 Custom Attributes
10. 使用属性(C# 编程指南)
11. 属性

用户:

特性的实现的功能的逻辑在哪里实现的？比如serializable特性的功能是允许序列化，那这个“允许序列化”是在哪里实现的？

元宝:

很好的问题——这正好戳到"特性"这个概念最容易让人迷糊的地方：**特性本身不实现功能，它只是个"标签"，真正干活的是"读这个标签的代码"。**

以 [Serializable] 为例拆开看

SerializableAttribute 这个类长这样（简化）：

```
namespace System
{
    [AttributeUsage(AttributeTargets.Class | AttributeTargets.Struct | AttributeTarget
s.Enum | AttributeTargets.Delegate)]
    public sealed class SerializableAttribute : Attribute
    {
        // 注意：构造器是空的，没有任何逻辑
        public SerializableAttribute() { }
    }
}
```

你贴 [Serializable]，编译器做的事就只是**在类型的元数据里写一条记录："这个 Type 被 SerializableAttribute 标记过"。**这个类本身一行业务逻辑都没有。

那"允许序列化"在哪实现的？——在**序列化器（Formatter）里。**

比如老牌的 BinaryFormatter（虽然现在不推荐用了，但逻辑最清楚）：

```
// BinaryFormatter.Serialize 内部大致逻辑
internal static void CheckSerializable(Type t)
{
    if (!t.IsSerializable)   // 本质就是去查元数据里有没有 [Serializable]
        throw new SerializationException(
            $"Type '{t}' is not marked as serializable.");
}
```

t.IsSerializable 底层走的其实就是 Attribute.IsDefined(t, typeof(SerializableAttribute))。**发现你贴了 → 继续走序列化流程；没贴 → 抛异常。**

所以链条是：

> 你贴 [Serializable] → 编译器写元数据 → BinaryFormatter 反射查元数据 → 发现标记 → 放行序列化

"允许"这两个字不是 SerializableAttribute 做的，是**序列化器看到这个标记后决定放行。**

推而广之：所有特性都是这个模式

特性	贴它的你	真正读它、干活的那一方
`[Serializable]`	业务代码	`BinaryFormatter` / `DataContractSerializer` 等
`[DllImport("xxx")]`	业务代码	**CLR 的 Interop 层**（加载 DLL、绑函数、marshal 参数）
`[Obsolete]`	业务代码	**C# 编译器**（编译期报 warning/error）
`[Conditional("DEBUG")]`	业务代码	**C# 编译器**（条件不满足就不把调用指令 emit 进 IL）
`[Test]` (xUnit/NUnit)	测试代码	**测试 Runner**（扫描程序集，找带 `[Test]` 的方法来跑）
`[Authorize]` (ASP.NET)	Controller/Action	**MVC 过滤器管道**（执行前检查登录/权限）
`[Table("Users")]` (EF)	实体类	**EF Core 的模型构建器**（把类映射到哪张表）

共性：`Attribute` 派生类本身 ≈ 纯数据结构（构造器参数 + 命名字段），逻辑零行。功能全在**"消费者"**那边——也就是框架/编译器/CLR/Runner，它们约定："我会在运行/编译期扫元数据，看到 X 标记就做 Y 事"。

这就引出一个重要视角

> 特性 = **编译期写入的声明性元数据** + **某种框架在合适时机读取它并执行对应逻辑**

所以你写自己的 `[MyAttribute]` 贴在某方法上，它本身啥也不会做——**除非你自己（或某个框架）在某个地方写 `GetCustomAttribute` 去读它，然后 decide 做什么。**这也是为什么 Richter 18.5 专门讲 `IsDefined` / `GetCustomAttribute` / `CustomAttributeData`——那是"消费者侧"的 API，没有这部分，特性就是死数据。

举个自制小例子把这条链闭环：

```
    // 1. 定义（纯数据，无逻辑）
    [AttributeUsage(AttributeTargets.Method)]
    public class AutoRunAttribute : Attribute { }

    // 2. 贴标签
    class Program
    {
        [AutoRun]
        static void Hello() => Console.WriteLine("跑了");

        static void Main()
        {
            // 3. 消费者：启动时任扫一遍，找到就 Invoke
            var methods = typeof(Program).GetMethods(BindingFlags.Static | BindingFlags.Non
    Public);
```

```
            foreach (var m in methods)
            {
                if (m.IsDefined(typeof(AutoRunAttribute), false))
                    m.Invoke(null, null);
            }
        }
    }
```

`[AutoRun]` 本身啥也不干，`Main` 里那段扫描代码才是"功能"所在。这跟 `[Serializable]` + `BinaryFormatter` 是同一个故事，只是规模小。

所以回到你的问题：**"允许序列化"不在 `SerializableAttribute` 里，在 `BinaryFormatter`（或任何序列化器）的实现里——它约定"见到这标签我才序列化"。** 理解到这一层，特性的整个设计哲学就通了：它是**声明式编程**的载体，把"做什么"和"怎么做"解耦——你只管贴标签声明意图，框架负责读标签执行逻辑。

这一层吃透的话，18.5 那节"检测定制特性"你读起来就会很顺——因为那就是"消费者侧"的 API。要继续往下拆 18.2 定义自己的 Attribute，还是先把 18.1 这个"谁实现"的模型再聊聊？

用户:

“18.3 特性构造器和字段/属性数据类型”这一节不太懂，你结合例子解释一下。

元宝:

这一节其实是在讲一个非常硬核的 CLR 限制：**特性最终是要被“烧录”进元数据的，而元数据不是堆，不能随便塞对象。**所以 C# 对你能写在 `[]` 里的东西卡得很死。

我按 Richter 的思路，拆成「规则 → 原因 → 例子」来讲。

一、核心结论先记住

> **特性构造器的实参 + 命名字段/属性的值，必须在编译期就能确定，并且必须是“元数据兼容类型”。**

换句话说：`[]` 里写的不是普通 C# 表达式，而是**元数据字面量**。

二、CLR 允许哪些“写进元数据”的类型？

Richter 列了一个白名单，你完全可以把它当成**“特性专用类型表”**：

分类	允许的具体类型
简单类型	`bool`, `byte`, `char`, `double`, `float`, `int`, `long`, `short`, `string`
枚举	任意 `enum`（本质是整型）
Type	`typeof(Foo)`（写入的是 TypeRef/TypeDef Token）
数组	**上述类型的数组**（如 `int[]`, `string[]`）

⚠️ 注意：这是**闭集**，不在表里的一律不行。

三、为什么会有这些限制？（关键）

1️⃣ 元数据里不存“对象”，只存“值”

特性信息存在 **Metadata Table** 里，不是 GC 堆里。

比如你写：

```
[MyAttr(10, "hello")]
```

编译器做的是：

- 把 `10` 写成一个 `Int32` 常量
- 把 `"hello"` 写成一个 `#US` (User String) 堆偏移
- 在 `CustomAttribute` 表里记一条记录

不会 new 一个对象，也不会执行任何代码。

👉 所以下面这种写法是非法的：

```
[MyAttr(DateTime.Now)]   // ❌ DateTime 不是常量
[MyAttr(new object())]   // ❌ new 表达式
[MyAttr(GetValue())]     // ❌ 方法调用
```

因为这些只能在**运行期**算出来，而特性必须在**编译期**定死。

2️⃣ `Type` 是个特例（非常重要）

这是本节最容易晕的点。

✅ **允许**：

```
[MyAttr(typeof(FileStream))]
```

原因是：

- `typeof(FileStream)` 在编译期就能解析
- 编译器写入的是 **TypeRef / TypeDef 元数据 token**
- 运行期 `GetCustomAttribute` 时，CLR 再用这个 token 解析出 `Type` 对象

❌ **不允许**：

```
[MyAttr(fileStream.GetType())] // ❌ 运行期才能知道
```

四、数组为什么可以？（**Richter** 特意讲的）

CLR 支持 **一维 0 下标数组（SZArray）**作为特性参数。

```
[AttributeUsage(AttributeTargets.Class)]
public class MyAttr : Attribute
{
    public int[] Values { get; }

    public MyAttr(int[] values)
    {
        Values = values;
    }
}

[MyAttr(new int[] { 1, 2, 3 })]
class Test { }
```

这里发生了什么？

- `{1,2,3}` 被编译成一个**静态只读数组**
- 元数据里保存的是数组元素的引用
- 运行期反序列化回来

⚠️ 但注意：**数组里的元素仍然必须遵守上面的白名单。**

五、枚举为什么可以？（这是个坑）

```
enum Color { Red, Blue }

[MyAttr(Color.Red)]
```

原因是：**枚举底层是整型常量**，编译期完全确定。

但 Richtere 提醒一点：

> 如果你改了枚举的定义（比如插入一个新值），**已编译好的特性不会自动更新。**

这是元数据绑定带来的副作用。

六、一个“合法 vs 非法”对照表（强烈建议记）

```
// ✅ 合法
[MyAttr(42)]
[MyAttr("abc")]
[MyAttr(typeof(string))]
[MyAttr(new int[] {1,2})]
[MyAttr(MyEnum.A)]

// ❌ 非法
[MyAttr(DateTime.Now)]
[MyAttr(new List<int>{1,2})]
[MyAttr(new Dictionary<string,int>())]
[MyAttr(new FileStream(...))]
[MyAttr(GetValue())]
```

七、为什么字段 / 属性也要遵守同样规则？

Richter 强调：**命名参数 = 构造完成后赋值。**

```
[DllImport("kernel32", CharSet = CharSet.Auto)]
```

等价于（概念上）：

```
var attr = new DllImportAttribute("kernel32");
attr.CharSet = CharSet.Auto;
```

如果这个赋值**不能在编译期完成**，元数据就写不进去。

所以：

- `public string Name { get; set; }` ✅
- `public DateTime Time { get; set; }` ❌（除非你永远不用它当特性参数）

八、用一句话总结这一节

> **特性的构造器和字段/属性，只能使用“编译期可表示为元数据常量”的类型；因为它们不是代码，而是被编译器序列化进元数据的声明性数据。**

这也正好呼应了你上一个问题：
👉 **特性本身不执行逻辑，只是"被读取的数据"**，而数据要能写进文件，就必须受限。

如果你愿意，下一步我可以帮你把 **18.4 检测特性（IsDefined vs GetCustomAttribute）**和这一节连起来讲——你会突然明白为什么 `GetCustomAttribute` 会"反序列化"特性，而不是 new 一个普通对象。要接着拆吗？

用户:

"18.4 检测定制特性"这一节不太懂。

元宝:

好，18.4「检测定制特性」这一节，正好和你上一问 18.3 是**对偶关系**：

- **18.3**：你写 `[Foo(1, "a")]`，编译器把构造器参数 + 命名字段**序列化**进元数据 → 限制一大堆（只能是常量/Type/数组…）
- **18.4**：运行期某段代码想"用"这个特性 → 得把元数据里的字节**反序列化**回 `Attribute` 实例 → 这就是"检测"干的事

Richter 这一节核心是给你讲清楚 **CLR 提供了哪几套 API 来"读"特性、各自代价和坑在哪。**

一、先建立画面：特性在元数据里是"死的"

贴完 `[Serializable]` 编译完，元数据 `CustomAttribute` 表里有一条记录，长这样（概念上）：

```
Parent: TypeDef → MyClass
Type: TypeRef → SerializableAttribute
Constructor Args: (empty)
Named Args: (empty)
```

它只是一个"配方"——`new SerializableAttribute()` 需要的参数都记着，但对象还没 new。

"检测特性" = **按配方把对象造出来（或者至少确认配方存在）。**

二、三套 API，Richter 按"从轻到重"给的

1️⃣ `IsDefined` —— 最轻，只"看一眼有没有"

```
bool Attribute.IsDefined(MemberInfo element, Type attributeType)
bool Attribute.IsDefined(MemberInfo element, Type attributeType, bool inherit)
```

行为：只查元数据表，**不构造 Attribute 对象。**

```
// 判断 MyClass 有没有 [Serializable]
bool ok = Attribute.IsDefined(typeof(MyClass), typeof(SerializableAttribute));
```

适合场景：**你只关心"有没有"，不关心参数值。**比如 `BinaryFormatter` 里那个 `CheckSerializable`，用 `IsDefined` 就够了，没必要 new 一个空对象的 `SerializableAttribute` 出来浪费。

> 💡 Richter 特意点了这个性能点：频繁扫特性（比如 MVC 启动扫全程序集 Controller），`IsDefined` 比 `GetCustomAttribute` 便宜，因为后者要真的 invoke 构造器、设命名字段。

---

2 `GetCustomAttribute(s)` —— 反序列化，造对象

```
Attribute Attribute.GetCustomAttribute(MemberInfo, Type)
Attribute[] Attribute.GetCustomAttributes(MemberInfo, Type, bool inherit)
```

行为：

1. 查元数据里有没有这条 `CustomAttribute` 记录
2. 有 → **按记录的构造器参数 new 实例**
3. 再按命名字段/属性赋值
4. 返回你这个 `Attribute` 对象

```
class AuthorAttribute : Attribute
{
    public string Name { get; }
    public AuthorAttribute(string name) => Name = name;
}

[Author("Jeffrey")]
class Book {}

var attr = (AuthorAttribute)Attribute.GetCustomAttribute(
    typeof(Book), typeof(AuthorAttribute));
Console.WriteLine(attr.Name); // "Jeffrey"
```

这一步你就明白 18.3 的限制为啥那么严了——**这里要真的调构造器**，`new AuthorAttribute("Jeffrey")`，所以构造器参数必须是元数据能表示的常量，否则反序列化不了。

---

3 `MemberInfo.GetCustomAttribute(s)` —— 实例方法版

```
var attrs = typeof(Book).GetCustomAttributes(typeof(AuthorAttribute), false);
```

Richter 提了一句：`System.Attribute` **的静态方法内部其实也是调** `MemberInfo` **的实例方法**，两套等价，看你顺手。

三、那个 `inherit` 参数（本节最容易踩的坑）

三个 API 都有个 `bool inherit`，默认 `false`。Richter 用不小篇幅讲这个。

规则一句话

> **类/方法/属性/事件** 上，`inherit: true` 会让 CLR **沿着继承链往上追**查基类有没有贴这个特性。

```
[Author("Base")]
class Base {}

class Derived : Base {}   // 自己没贴

// inherit = false → 找不到
Attribute.GetCustomAttribute(typeof(Derived), typeof(AuthorAttribute), false); // null

// inherit = true → 追到 Base，找到了
Attribute.GetCustomAttribute(typeof(Derived), typeof(AuthorAttribute), true);  // Base 那个
```

但有三个坑 **Richter** 点名了

坑 1：`[AttributeUsage(Inherited = false)]` 会阻断

```
[AttributeUsage(AttributeTargets.Class, Inherited = false)]
class AuthorAttribute : Attribute { ... }

[Author("Base")]
class Base {}
class Derived : Base {}

Attribute.GetCustomAttribute(typeof(Derived), typeof(AuthorAttribute), true); // null！
```

虽然 `inherit=true` 说"往上追"，但特性类自己声明了 `Inherited=false` → **这条特性拒绝被继承。**

坑 2：不是所有目标都能"继承"

- 能追：`Type`（类继承）、`Method`（virtual/override）、`Property`、`Event`
- 不能追：`Field`（CLR 里字段不参与继承语义）、`Assembly`、`Module`、`ReturnValue`、`Param`

对 `Field` 传 `inherit:true` 也没用，CLR 直接忽略。

坑 3：方法 override 场景的"覆盖"语义

```
class Base
{
    [Obsolete]
    virtual void M() { }
}

class Derived : Base
{
    [Obsolete("new msg")]   // AllowMultiple=false，默认覆盖基类那份
    override void M() { }
}
```

`inherit=true` 在 `Derived.M` 上查，拿到的是 **Derived 自己贴的那份**（新 msg），基类的被"盖掉"——因为 `Obsolete` 的 `AllowMultiple = false`。如果 `AllowMultiple=true`（比如某些框架自己定义的），那基类的 + 派生类的会**叠加**返回数组。

四、`CustomAttributeData` —— 本节收尾的安全 API

Richter 这一节最后讲这个，动机是**"反序列化特性可能触发代码执行 + 加载依赖类型"**。

问题在哪？

`GetCustomAttribute` 内部要做：

1. 解析构造器参数里的 `typeof(X)` → **得加载 X 这个 Type**（可能触发程序集加载）
2. `new AttributeXXX(args)` → **执行构造器代码**（特性作者可以在构造器里写任意逻辑，包括抛异常、访问文件、调非托管…）

在**沙箱 / 只加载元数据不加载执行代码**的场景下（比如工具、反编译、部分信任），这不行。

`CustomAttributeData` 的解法

```
IList<CustomAttributeData> data = CustomAttributeData.GetCustomAttributes(typeof(Boo
k));
foreach (var d in data)
{
    Console.WriteLine(d.AttributeType.FullName);        // 特性类名
    Console.WriteLine(d.ConstructorArguments[0]);       // 构造器参数（原始值）
    foreach (var na in d.NamedArguments)
        Console.WriteLine(na.MemberName + "=" + na.TypedValue.Value);
}
```

关键区别：

	GetCustomAttribute	CustomAttributeData
new 实例?	✅ 是，调构造器	❌ 否，只给"配方"
加载依赖 Type?	✅ 会（typeof 参数要解析）	❌ 不会，Type 以 `TypeHandle` 形式懒表示
执行特性作者代码?	✅ 会	❌ 不会

所以 Richter 说：**只关心"贴了什么、参数是什么"，不打算真用这个 Attribute 对象 → 用 `CustomAttributeData` 更安全更轻。**

五、把 18.3 + 18.4 串成一句完整的话

> 18.3 你写 `[Foo(1, typeof(Bar))]` → 编译器把 `1` 和 `Bar` 的 TypeRef **序列化**进元数据 `CustomAttribute` 表；
> 18.4 你调 `GetCustomAttribute` → CLR 按表里的配方 **反序列化**：加载 `Bar`、`new Foo(1)`、赋值命名字段 → 返回一个活对象。
> `IsDefined` 是"只查表不配方"，`CustomAttributeData` 是"只给配方不动手"。

六、回扣你上一问的 `AutoRun` 例子

```
if (m.IsDefined(typeof(AutoRunAttribute), false))   // 18.4 的 API
    m.Invoke(null, null);
```

这里用 `IsDefined` 就够——因为我们只要"有没有"，不需要读 `AutoRunAttribute` 的参数（它本来也没参数）。如果哪天你改成：

```
[AutoRun(Order = 10)]
```

那就要 `GetCustomAttribute` 把 `Order` 读出来排序了——这就是"消费者"侧需求决定你用哪套 API。

这一节吃透的标志是：**看到 `IsDefined` / `GetCustomAttribute` / `CustomAttributeData` 能立刻反应出"查表 / 反序列化 / 只拿配方"这三档代价。**

下一节 18.5 是「条件特性类 `ConditionalAttribute`」，那个跟编译期 `#if` 容易混，要接着拆吗？还是 18.4 里 `inherit` 那个追查逻辑想再抠抠（比如 virtual/override 方法上特性的叠加规则，那个书里例子挺细的）？

用户:

typeof()返回的不是type类型吗？为什么上面提到的有的方法参数可以用MemberInfo 来对接typeof？

元宝:

这是一个**非常棒的问题**，说明你已经不是在"背 API"，而是在琢磨 .NET 类型系统的设计了 👍
答案是：`Type` **本身就是** `MemberInfo` **的子类。**

一、先看继承关系（关键）

在 .NET 反射体系中，有一个清晰的继承链：

```
Object
 └─ MemberInfo          （抽象基类：所有“成员”的抽象）
     ├─ Type            （类型本身）
     ├─ MethodBase
     │   ├─ MethodInfo
     │   └─ ConstructorInfo
     ├─ PropertyInfo
     ├─ FieldInfo
     ├─ EventInfo
     └─ ...
```

👉 **Type : MemberInfo**

所以这句话在 C# 里是完全合法的：

```
MemberInfo mi = typeof(string);
```

二、为什么“检测特性”的方法都用 **MemberInfo** 做参数？

Richter 在 18.4 里大量出现这样的签名：

```
bool Attribute.IsDefined(MemberInfo element, Type attributeType)
Attribute Attribute.GetCustomAttribute(MemberInfo element, Type attributeType)
```

而不是写成：

```
bool IsDefined(Type type, Type attributeType)
bool IsDefined(MethodInfo method, Type attributeType)
bool IsDefined(PropertyInfo prop, Type attributeType)
```

原因：**一套 API，通吃所有可贴特性的目标**

特性可以贴在：

目标	对应的反射类型
class / struct / enum	`Type`
method	`MethodInfo`
constructor	`ConstructorInfo`
property	`PropertyInfo`
field	`FieldInfo`
event	`EventInfo`
parameter	`ParameterInfo`
assembly	`Assembly`（❗不是 MemberInfo）
module	`Module`（❗不是 MemberInfo）

CLR 设计者选择：

- **凡是“属于某个类型的成员”的特性 → 用 `MemberInfo` 抽象**
- `Assembly` / `Module` 比较特殊，单独提供重载

所以你看到的这些方法，本质上是**面向“成员”的通用接口。**

三、那 `typeof()` 返回的是什么？

```
Type t = typeof(String);
```

- `typeof(String)` → **编译期运算符**
- 返回值类型：`Type`
- `Type` → 继承自 `MemberInfo`

于是你可以直接传进去：

```
// 完全合法
bool b = Attribute.IsDefined(typeof(MyClass), typeof(SerializableAttribute));
```

在调用时发生了一次**隐式向上转型**：

```
Type → MemberInfo
```

就像：

```
object o = "hello";   // string → object
MemberInfo mi = typeof(MyClass); // Type → MemberInfo
```

四、一个对比示例，立刻就通了

1️⃣ 贴在类上（用 `Type`）

```
[Serializable]
class Foo { }

// Type → MemberInfo
bool ok = Attribute.IsDefined(typeof(Foo), typeof(SerializableAttribute));
```

2️⃣ 贴在方法上（用 `MethodInfo`）

```
class Bar
{
    [Obsolete]
    public void M() { }
}

MethodInfo mi = typeof(Bar).GetMethod("M");
bool ok = Attribute.IsDefined(mi, typeof(ObsoleteAttribute));
```

3️⃣ 统一写法（体现设计思想）

```
void PrintIfHasAttribute(MemberInfo member)
{
    if (Attribute.IsDefined(member, typeof(DescriptionAttribute)))
        Console.WriteLine(member.Name + " has Description");
}

PrintIfHasAttribute(typeof(Foo));          // Type
PrintIfHasAttribute(mi);                   // MethodInfo
PrintIfHasAttribute(typeof(Bar).GetProperty("P")); // PropertyInfo
```

同一个方法，处理所有成员类型 —— 这就是 `MemberInfo` 存在的意义。

五、那 `Assembly` 和 `Module` 为什么不是 `MemberInfo`？

这是很多人第二次卡住的地方。

- `Assembly`：整个 EXE/DLL
- `Module`：一个程序集里的物理模块（大多只有一个 `Module`）

- `MemberInfo`：**“类型内部的成员”**

它们不属于“类型内部的成员”，所以：

- 不继承 `MemberInfo`
- 有自己的独立 API：

```
[assembly: AssemblyVersion("1.0.0")]

// 不是 MemberInfo
Assembly asm = Assembly.GetExecutingAssembly();
Attribute.IsDefined(asm, typeof(AssemblyVersionAttribute));
```

Richter 在书里其实也暗示了这一点：
👉 `assembly:` / `module:` 前缀是**元数据层面的顶级目标**，和 `class/method/field` 不在同一层级。

六、用一句话总结你这个问题

> **`typeof()` 返回的是 `Type`，而 `Type` 继承自 `MemberInfo`，因此凡是接收 `MemberInfo` 的“检测特性”方法，都可以直接用 `typeof(...)` 作为参数。**

这也是 .NET 反射 API 的一个典型设计模式：
用抽象基类（MemberInfo）统一行为，用派生类（Type / MethodInfo / PropertyInfo）表达具体差异。

如果你愿意，下一步我们可以把 `MemberInfo` **vs** `ICustomAttributeProvider` 这个更底层的接口设计也拆一下（Richter 没明写，但很多高级反射代码会碰到），或者继续把 18.5 的 `ConditionalAttribute` 和编译期决策逻辑串起来。你想往深走还是往宽走？

用户:

文章中这一段不太懂，请结合例子解释：

“System.Reflection 命名空间定义了几个类允许检查模块的元数据。这些类包括 Assembly，Module，ParameterInfo，MemberInfo，Type，MethodInfo，Type，MethodInfo，ConstructorInfo，FieldInfo，EventInfo，PropertyInfo及其各自的 *Builder 类。所有类都提供了 IsDefined 和 GetCustomAttributes方法。”

元宝:

这段话其实是 Richter 在帮你**“画地图”**——他在告诉你：

> **凡是“能贴特性的地方”，CLR 都给它配了一套反射类，而且这些类统统能查特性。**

你之所以觉得乱，是因为一下子抛出了一堆类型名。我们把它拆开，用**“角色 + 例子”**来理解。

一、先给结论（最重要的一句话）

> 这些类，本质上都是“元数据载体”。
> 它们要么实现了 `ICustomAttributeProvider` 接口，要么内部封装了同样的机制，因此全部提供了 `IsDefined` 和 `GetCustomAttributes` 方法。

也就是说：
能贴 `[Attribute]` 的地方 → 就有办法“读”这个 Attribute。

二、把这些类分成三组，立刻清晰

✅ 第一组：最常见，继承自 `MemberInfo`

类	对应代码里的东西	例子
`Type`	class / struct / enum / interface	`typeof(MyClass)`
`MethodInfo`	方法	`void M()`
`ConstructorInfo`	构造函数	`MyClass()`
`PropertyInfo`	属性	`int Age { get; set; }`
`FieldInfo`	字段	`int x;`
`EventInfo`	事件	`event Action E;`
`ParameterInfo`	参数	`(int x, string y)`

它们的关系：

```
MemberInfo
 ├─ Type
 ├─ MethodBase
 │   ├─ MethodInfo
 │   └─ ConstructorInfo
 ├─ PropertyInfo
 ├─ FieldInfo
 └─ EventInfo
```

👉 **它们都来自 `System.Reflection`**
👉 **它们都“属于某个类型内部”**

✅ 第二组：程序集级（不在 **MemberInfo** 树下）

类	对应	例子
`Assembly`	整个 DLL / EXE	`[assembly: AssemblyVersion("1.0")]`
`Module`	模块（一个程序集通常一个）	`[module: ...]`

它们**不是** `MemberInfo`，但它们**也能贴特性**，所以也有 `IsDefined` / `GetCustomAttributes`。

✅ 第三组：*Builder 系列（高级 / Emit）

类	用途
`AssemblyBuilder`	运行时创建程序集
`ModuleBuilder`	创建模块
`TypeBuilder`	创建类型
`MethodBuilder`	创建方法
`PropertyBuilder`	创建属性
`FieldBuilder`	创建字段
`ConstructorBuilder`	创建构造函数
`EventBuilder`	创建事件
`ParameterBuilder`	创建参数

👉 这些都在 `System.Reflection.Emit`
👉 用于 **动态生成代码 / 运行时拼程序集**
👉 设计上要和"读元数据"对称：**能写特性，也能查特性**

三、用一个完整示例把它们串起来

1️⃣ 各种地方贴特性

```
using System;
using System.Reflection;

[assembly: Description("MyAssembly")]
[module: Description("MyModule")]

[Description("MyClass")]
class MyClass
{
```

```
    [Description("MyField")]
    public int x;

    [Description("MyMethod")]
    public void M([Description("MyParam")] int p) { }
}
```

2️⃣ 用不同类去“检测”这些特性

✅ Assembly

```
Assembly asm = Assembly.GetExecutingAssembly();
bool b1 = asm.IsDefined(typeof(DescriptionAttribute), false);
```

✅ Module

```
Module mod = asm.Modules.First();
bool b2 = mod.IsDefined(typeof(DescriptionAttribute), false);
```

✅ Type

```
Type t = typeof(MyClass);
bool b3 = t.IsDefined(typeof(DescriptionAttribute), false);
```

✅ Field

```
FieldInfo f = t.GetField("x");
bool b4 = f.IsDefined(typeof(DescriptionAttribute), false);
```

✅ Method + Parameter

```
MethodInfo m = t.GetMethod("M");
bool b5 = m.IsDefined(typeof(DescriptionAttribute), false);

ParameterInfo p = m.GetParameters()[0];
bool b6 = p.IsDefined(typeof(DescriptionAttribute), false);
```

👉 **同一个方法名 `IsDefined`，用在完全不同的对象上**
👉 这就是 Richter 说“所有类都提供了这些方法”的真正含义

四、为什么“所有类都提供这些方法”？

根本原因：`ICustomAttributeProvider`

虽然 Richter 没在这段明说，但底层真相是：

```
public interface ICustomAttributeProvider
{
    object[] GetCustomAttributes(bool inherit);
    object[] GetCustomAttributes(Type attributeType, bool inherit);
    bool IsDefined(Type attributeType, bool inherit);
}
```

所有反射类（`Assembly` 除外，但它内部实现类似契约）都实现了这个接口。

👉 **这是 CLR 的统一设计：**

- 元数据表里能贴 Attribute 的地方
- 反射层就给你一个对象来表示它
- 这个对象就一定支持“查特性”

五、那 `*Builder` 为什么也算？

因为 **Emit = 写元数据**，而特性是元数据的一部分。

```
AssemblyBuilder ab = AssemblyBuilder.DefineDynamicAssembly(...);
ab.SetCustomAttribute(new CustomAttributeBuilder(...));
bool b = ab.IsDefined(typeof(DescriptionAttribute), false);
```

👉 你**写进去的特性**
👉 马上就能**查出来**
👉 这样才能保证“读写一致”

六、把这段话翻译成人话

> *“System.Reflection 命名空间定义了很多类，用来读取模块里的元数据。
> 不管你贴特性在程序集、模块、类型、方法、字段、属性、事件还是参数上，
> CLR 都会给你一个对应的反射对象（Assembly / Type / MethodInfo 等），
> 而且这些对象全都支持 `IsDefined` 和 `GetCustomAttributes`，
> 甚至你在运行时用 Builder 动态创建类型时也一样。”*

七、和你前面问题的完美呼应

你之前问过：

> `typeof()` 返回 `Type`，为什么能传给 `MemberInfo`？

现在答案更完整了：

- `Type` 是 `MemberInfo`
- `MemberInfo` 是 **“能贴特性的成员”的抽象**
- 所有“能贴特性的东西” → 都有 `IsDefined` / `GetCustomAttributes`
- 所以：**只要是反射对象，几乎都能查特性**

八、一张“心智模型图”（建议记住）

```
元数据位置          → 反射类                → 能做的事
-------------------------------------------------
Assembly          → Assembly             → 查/写特性
Module            → Module               → 查/写特性
Type              → Type                 → 查/写特性
Method            → MethodInfo           → 查/写特性
Field             → FieldInfo            → 查/写特性
Property          → PropertyInfo         → 查/写特性
Event             → EventInfo            → 查/写特性
Parameter         → ParameterInfo        → 查/写特性
（动态创建）       → *Builder             → 查/写特性
```

如果你愿意，下一步我可以帮你把 **`Attribute.GetCustomAttribute`（静态）vs `MemberInfo.GetCustomAttributes`（实例）**的调用偏好讲清楚，或者带你把 **`CustomAttributeData` 在这些类上的用法** 补一刀 —— 这在写 AOP / 框架 / 分析工具时非常实用。

用户:

“18.6 检测定制特性时不创建从 Attribute 派生的对象”，这一节不太懂。

元宝:

好，18.6 这个标题本身就藏着重点——**"检测……但不创建 Attribute 对象"**，这正是和 18.4 的 `GetCustomAttribute` 对着来的。

我先把这一节的动机和手段拆开，再给你一个能跑的思路。

一、先回顾 18.4 的 `GetCustomAttribute` 做了什么

```
[Author("Jeffrey")]
class Book {}

var attr = (AuthorAttribute)Attribute.GetCustomAttribute(
    typeof(Book), typeof(AuthorAttribute));
// ↑ 这一步发生了什么？
```

Richter 在 18.4 讲过，这一步是**反序列化**：

1. 查元数据 `CustomAttribute` 表
2. 找到 `AuthorAttribute` 的构造器
3. **加载 `AuthorAttribute` 所在的程序集**
4. **`new AuthorAttribute("Jeffrey")`** ← 构造器真的被执行
5. 再把命名字段/属性赋值

👉 **对象真的被 new 出来了，构造器代码真的跑了。**

二、那 18.6 问的就是：这有什么问题？

Richter 在这一节点出了**三个风险场景**，这就是 18.6 存在的理由。

⚠️ 风险 1：构造器里可能有"坏代码"

特性类是你（或别的库）写的，构造器里**可以写任意 C#**：

```
class EvilAttribute : Attribute
{
    public EvilAttribute()
    {
        File.Delete("C:\\important.txt");   // 构造器里删文件
        throw new Exception("boom");         // 或抛异常
    }
}
```

你只是想"检测一下有没有这个特性"，结果：

- `GetCustomAttribute` → new 实例 → **构造器执行** → **文件没了**

⚠️ 风险 2：`typeof(X)` 参数会触发程序集加载

回忆 18.3 的合法参数：

```
[MyAttr(typeof(FileStream))]
```

`GetCustomAttribute` 反序列化时：

- 构造器参数里有 `typeof(FileStream)`
- CLR **必须把** `System.IO.FileStream` **所在的** `System.Runtime` / `mscorlib` **加载进来**
- 在只"看元数据"的场景下，这属于**过度加载**

⚠️ 风险 3：`ReflectionOnlyLoad` 场景会直接崩

这是 18.6 **最核心的语境。**

CLR 提供过一个 API：

```
    Assembly asm = Assembly.ReflectionOnlyLoadFrom("Foo.dll");
```

意思是：**只把程序集的元数据加载进来，不加载任何可执行代码（IL 不 JIT、不执行）。**

典型用途：

- 反编译工具
- 静态分析
- 插件扫描（只看声明，不真的加载插件进当前 AppDomain）

在这个上下文里：

```
    // ReflectionOnly 加载的 Assembly
    var attrs = Attribute.GetCustomAttribute(type, typeof(MyAttr));
    // ❌ 可能直接失败：
    //   - 要 new MyAttr → 要执行 IL → ReflectionOnly 不允许
    //   - typeof(X) 参数 → 要解析 Type → 可能触发非 ReflectionOnly 加载
```

👉 `GetCustomAttribute` **在 ReflectionOnly 场景下"太重了，用不了"。**

三、18.6 给出的解法：`CustomAttributeData`

Richter 这一节的主角就是这个类（18.4 收尾其实已经露过脸，18.6 是正式展开）。

```
    IList<CustomAttributeData> list =
        CustomAttributeData.GetCustomAttributes(typeof(Book));
```

它和 `GetCustomAttribute` 的关键区别

维度	GetCustomAttribute	CustomAttributeData
new 实例?	✅ 是，调构造器	❌ 否
执行特性作者代码?	✅ 会	❌ 不会
加载依赖程序集?	✅ 会（typeof 参数）	❌ 不会
ReflectionOnly 能用?	❌ 不行	✅ 可以
拿到什么?	`Attribute` 对象	"配方"数据结构

四、`CustomAttributeData` 长什么样（关键 API）

```
var datas = CustomAttributeData.GetCustomAttributes(typeof(Book));
foreach (var d in datas)
{
    Console.WriteLine(d.AttributeType.FullName);  // 特性类名

    foreach (var arg in d.ConstructorArguments)
        Console.WriteLine($"  构造器参数: {arg.Value} (类型={arg.ArgumentType})");

    foreach (var na in d.NamedArguments)
        Console.WriteLine($"  {na.MemberName} = {na.TypedValue.Value}");
}
```

还是用 `[Author("Jeffrey")]` 举例，输出大概是：

```
AuthorAttribute
  构造器参数: Jeffrey (类型=System.String)
```

注意：**`d.ConstructorArguments[0].Value` 是 `"Jeffrey"` 这个 `string` 常量，不是 `AuthorAttribute` 实例。**

五、用一个"ReflectionOnly"的完整例子把这节闭环

这是 18.6 最经典的演示场景。

```
// 1. 只加载元数据，不加载可执行代码
Assembly asm = Assembly.ReflectionOnlyLoadFrom("Foo.dll");

Type t = asm.GetType("MyClass");

// 2. ❌ 这条路走不通
// Attribute.GetCustomAttribute(t, typeof(SerializableAttribute));
```

```
//     → 要 new 对象 → ReflectionOnly 禁止

// 3. ✅ 18.6 给的路
IList<CustomAttributeData> list = CustomAttributeData.GetCustomAttributes(t);
foreach (var d in list)
{
    // d.AttributeType 本身也是在 ReflectionOnly 上下文里的 Type
    // 但没关系，我们只是读名字
    Console.WriteLine(d.AttributeType.FullName);
}
```

👉 **全程没有 new 任何一个 Attribute，没有执行一行特性作者的代码，没有触发额外程序集加载** —— 这就是 18.6 标题的含义。

六、18.4 vs 18.6 怎么选？（Richter 给的决策）

Richter 没有明说"决策树"，但这一章读下来可以总结出：

> **你要"用"这个特性（读它的属性、根据它做分支）→ `GetCustomAttribute`**
> **你只"看"这个特性（扫描、分析、ReflectionOnly、工具场景）→ `CustomAttributeData`**

更细一档：

你的需求	用哪个
只想知道"有没有"	`IsDefined`（18.4，最轻）
要读参数值、要用这个对象	`GetCustomAttribute`（18.4）
沙箱 / ReflectionOnly / 不想执行构造器	`CustomAttributeData`（18.6）
扫描全程序集做静态分析	`CustomAttributeData`（18.6）

七、把 18.3 → 18.4 → 18.6 串成一条完整链路

> **18.3**：你写 `[Foo(1, typeof(Bar))]` → 编译器把 `1` + `Bar` 的 TypeRef **序列化**进元数据
> **18.4**：`GetCustomAttribute` → **反序列化**：加载 Bar、`new Foo(1)` → 返回活对象
> **18.6**：`CustomAttributeData` → **只把配方读出来**：`ConstructorArguments = [1]`，`AttributeType = Foo` → 不 new、不加载、不执行

三档代价，三套 API，对应三种场景。

八、一个容易混的点（顺手提）

`CustomAttributeData` 读到的 `ConstructorArguments[i].Value`：

- 如果是 `int` / `string` / `enum` → 直接是值
- 如果是 **`typeof(X)`** → 返回的是 **`Type` 对象**，但这个 `Type` 是**懒的**，不会触发 X 的程序集加载（在 ReflectionOnly 下尤其明显）

所以即使特性写了 `[MyAttr(typeof(FileStream))]`，`CustomAttributeData` 也能安全读，不会把 `System.IO` 拉进来——这就是它"安全"的核心。

这一节吃透的标志是：**看到 `CustomAttributeData` 能立刻反应出"ReflectionOnly / 沙箱 / 静态分析"这三个词。**

18.6 之后 18.7 是 `ConditionalAttribute` 对吧？那个是编译期决策，和这一章其他"运行期反射"的路子完全不一样，要不要接着拆？还是 18.6 里 `CustomAttributeData` 的 `ConstructorArguments` / `NamedArguments` 那些细字段想再抠一下？

用户:

“18.7 条件特性类”不太懂。

元宝:

好，18.7「条件特性类」这一节其实是个**"编译期魔法"，跟前面 18.4 / 18.6 那种"运行期反射查特性"的路子**完全不是一个世界**。很多人卡住是因为把 `ConditionalAttribute` 和 `#if` 搞混了，或者没分清它贴在"方法"上 vs 贴在"特性类"上有啥区别。

我按 Richter 的思路拆。

一、先认清 `ConditionalAttribute` 是谁

```
namespace System.Diagnostics
{
    [AttributeUsage(AttributeTargets.Method | AttributeTargets.Class, Inherited = fals
e)]
    public sealed class ConditionalAttribute : Attribute
    {
        public ConditionalAttribute(string conditionString) { ... }
    }
}
```

它有两个合法目标：

- **方法**（最常见，大家熟的 `Debug.Write` 那套）
- **类**（且这个类必须派生自 `Attribute`）→ **这就是 18.7 的主题**

二、先复习：贴在"方法"上是啥效果（帮助理解）

```
[Conditional("DEBUG")]
static void Log(string s) => Console.WriteLine(s);

// 调用处
Log("hello");
```

编译期发生的事：

如果 `DEBUG` 符号**没定义** → C# 编译器**根本不把 `Log("hello")` 这条 call 指令 emit 进 IL。**

等价于（编译期自动变成）：

```
// 非 DEBUG 下，上面那行直接消失，连 IL 都没有
```

👉 **方法本体还在程序集里**（你能反射调到），但**所有调用点被编译器删了。**

三、18.7 的重点：贴在"特性类"上

1️⃣ 写法

```
[Conditional("DEBUG")]
[AttributeUsage(AttributeTargets.Method)]
public class TestAttribute : Attribute
{
    public TestAttribute() { Console.WriteLine("TestAttribute ctor"); }
}
```

2️⃣ 使用

```
class Program
{
    [Test]
    static void M() { }
}
```

3️⃣ 编译期发生的事（关键！）

> **`Conditional` 贴在特性类上 ≠ 控制"特性的行为"，而是控制"这个 `[Test]` 会不会被写进元数据"。**

分两种情况：

✅ 定义了 `DEBUG`（比如 Debug 配置）

编译器正常把 `[Test]` 写进 `CustomAttribute` 表 → 运行期 `IsDefined` / `GetCustomAttribute` 能查到 → 构造器执行。

❌ 没定义 `DEBUG`（比如 **Release** 配置）

编译器**直接跳过这个特性 → 元数据里根本没有这条** `CustomAttribute` **记录。**

效果等同于：你源码里写了 `[Test]`，但**编译器当它不存在。**

四、用一段完整代码看差异

```
#define DEBUG
// 把这行注释掉切到 Release 语义

[Conditional("DEBUG")]
class TestAttribute : Attribute
{
    public TestAttribute() => Console.WriteLine("ctor run");
}

[Test]
class MyClass { }

class Program
{
    static void Main()
    {
        var ok = Attribute.IsDefined(typeof(MyClass), typeof(TestAttribute));
        Console.WriteLine(ok);  // DEBUG 开 → True；关 → False
    }
}
```

配置	元数据里有 `[Test]`？	`IsDefined`	构造器执行?
DEBUG 开	✅ 有	True	✅ 执行
DEBUG 关	❌ 没有	False	❌ 不执行

👉 **注意**：不是"有特性但跳过"，是**元数据里压根没写这条**——这跟 18.4/18.6 那套运行期逻辑是两码事。

五、**18.7** 这一节 **Richter** 想表达的"核心动机"

场景：测试框架 **/** 诊断特性

Richter 举的思路大概是这样：

```
// 单元测试框架的内部
[Conditional("TEST")]
public class TestAttribute : Attribute { ... }

[Conditional("TEST")]
public class ExpectedExceptionAttribute : Attribute { ... }
```

你写测试：

```
[Test]
public void Calculator_Add_Works()
{
    Assert.AreEqual(3, Calc.Add(1, 2));
}
```

- **TEST 符号开** → 特性写进元数据 → 测试 Runner 扫到 → 跑测试
- **TEST 符号关（Release 发生产）** → 特性不进元数据 → Runner 扫不到 → 测试代码还在方法体里但没人调它（方法本身也可加 `[Conditional("TEST")]` 让调用也消失）

👉 **发布到生产时，不需要 `#if` 删代码，也不需要担心"特性拖累反射扫描"——编译器直接没写进去，干净。**

六、跟 `#if DEBUG` 的区别（这一节最容易混）

很多人以为：

```
#if DEBUG
[Test]
#endif
class MyClass {}
```

跟

```
[Conditional("DEBUG")]
class TestAttribute : Attribute {}

[Test]
class MyClass {}
```

效果一样？**不一样，而且差别很大。**

维度	`#if DEBUG`	`[Conditional("DEBUG")]` 在特性类上
作用阶段	预处理（比编译还早）	编译期（C# 编译器 emit metadata 时）
特性类本身	还在程序集里	还在程序集里
`[Test]` 使用点	没 `#if` 保护的那边 → 编译报错（找不到 TestAttribute）	源码里 `[Test]` 始终合法，编译器决定是否写元数据
多个条件	难搞（嵌套 `#if`）	可叠多个 `[Conditional]`

叠多个 `[Conditional]` 这个要单说：

```
[Conditional("DEBUG")]
[Conditional("TEST")]
class TestAttribute : Attribute {}
```

语义是 **OR**：`DEBUG` 或 `TEST` 任一定义 → 特性写进元数据。（方法上的 `[Conditional]` 叠多个也是 OR，这是 ECMA-335 规定的。）

七、跟 18.4 / 18.6 的呼应（这一章的收口）

把 18.7 放回第 18 章全貌看：

节	维度	干了啥
18.4 `GetCustomAttribute`	运行期	反序列化，new 对象
18.6 `CustomAttributeData`	运行期	只拿配方，不 new
18.7 `Conditional`	**编译期**	**根本不把特性写进元数据**

👉 18.7 是**最"前置"的一刀**：编译期就砍掉，运行期连"查"的机会都没有。

这也解释了为什么 Richter 把 18.7 放最后一节——它是特性系统里**唯一一个"编译器参与决策"**的机制，其他全是 CLR + 反射的活。

八、一个坑（Richter 没明说但值得知道）

`[Conditional]` 贴在特性类上时：

- 特性类**自身**还在程序集里，能被 `typeof(TestAttribute)` 拿到
- 但**别人贴的 `[Test]` 有没有进元数据**，取决于条件符号
- 所以你写框架时，**不能假设"用户贴了我就一定能扫到"**——Release 下扫不到是正常的

测试框架（xUnit/NUnit/MSTest）实际上**很少用 `[Conditional("TEST")]` 这么玩**，因为它们希望 Release 也能跑测试。这个机制更适合**"开发期诊断特性"**——比如你自己写的 `[AutoTracing]`、`[DumpToLog]`，只在 DEBUG 生效，发生产自动消失，不发警告不报错，干净。

九、一句话收 18.7

> **`[Conditional("XXX")]` 贴在特性类上 = 告诉 C# 编译器："只有当 XXX 条件符号定义时，才把别人贴的这个特性写进元数据；否则当没看见。"**
> 这是编译期裁剪，不是运行期跳过——所以 18.4 的 `IsDefined` / 18.6 的 `CustomAttributeData` 在条件不满足时，连"查"都查不到，因为元数据里根本没这条。

第 18 章到 18.7 就收完了。整体脉络是：

> **18.1 怎么用 → 18.2 怎么定义 → 18.3 参数限制（元数据能存啥）→ 18.4 运行期查（反序列化）→ 18.6 运行期查但不 new（安全/ReflectionOnly）→ 18.7 编译期裁（Conditional）**

这一章的精髓其实就是一句话：**特性 = 编译期写进元数据的配方 + 某处代码去读它**；18.7 是把"写"那一步加了条件。

要不要我帮你把整章用一张"什么时候用哪套 API"的速查表收个尾？或者你有哪一节还想再抠细的？

用户:

“六、跟 #if DEBUG的区别（这一节最容易混）”，你说的这一部分不太懂。

元宝:

好，这块确实是 18.7 最容易晕的，因为**表面效果像，底层机制完全两码事。**我用一个最小对照把差异打穿。

一、先给一句核心区分

> **`#if DEBUG` 是"预处理器"干的，砍的是"源码行"；**
> **`[Conditional("DEBUG")]` 是"C# 编译器"干的，砍的是"元数据记录 / 调用指令"。**

预处理器比编译器还早，它根本不知道什么叫 `Attribute`、什么叫类——它就是个**文本裁剪器。**

二、用同一段需求对比两种写法

需求：我希望 `[Test]` 这个特性**只在 DEBUG 下生效，Release 下悄无声息消失。**

写法 A：`#if DEBUG`（预处理方案）

```
#define DEBUG
// 把这行注释掉 = Release

#if DEBUG
[AttributeUsage(AttributeTargets.Method)]
public class TestAttribute : Attribute
```

```
{
    public TestAttribute() => Console.WriteLine("TestAttribute ctor");
}
#endif

#if DEBUG
[Test]
#endif
static void M() { }

// 如果上面 TestAttribute 被 #if 砍了，下面这行 Release 下直接编译报错：
// [Test]  // ❌ CS0246：找不到类型或命名空间 TestAttribute
```

预处理阶段发生的事（编译器还没起床）：

预处理器按 `#define` 有没有，决定**哪些文本行留、哪些行删**：

- DEBUG 开 → `#if DEBUG` 块内文本保留 → 编译器能看到 `class TestAttribute` + `[Test]`
- DEBUG 关 → `#if DEBUG` 块内文本**整块删掉** → 编译器根本不知道世界上有 `TestAttribute` 这个类

👉 后果：

- Release 下 `class TestAttribute` 不存在
- 你源码里如果还有 `[Test]`（忘了包 `#if`），**编译直接报错 CS0246**
- 你必须**每一个 `[Test]` 使用点都包 `#if DEBUG`**，否则不配套

写法 B：`[Conditional("DEBUG")]` 贴在特性类上（18.7 方案）

```
//#define DEBUG   // 关掉 = Release

[Conditional("DEBUG")]
[AttributeUsage(AttributeTargets.Method)]
public class TestAttribute : Attribute
{
    public TestAttribute() => Console.WriteLine("TestAttribute ctor");
}

// 注意：这里【没包 #if】
[Test]
static void M() { }
```

编译期发生的事：

- `class TestAttribute` **始终被编译器看到** → 它在程序集里存在，`typeof(TestAttribute)` 能拿到
- 编译器处理 `[Test]` 这一行时，发现 `TestAttribute` 身上有 `[Conditional("DEBUG")]`

- DEBUG 开 → 把 `[Test]` **写进元数据** `CustomAttribute` 表 ✅
- DEBUG 关 → **跳过这条特性，不写进元数据** ❌（但源码这行合法，不报错）

👉 后果：

- `TestAttribute` 类**永远存在**
- 用户源码里 `[Test]` **永远合法**，不用每个使用点包 `#if`
- 只是 Release 下元数据里**没有这条记录** → 运行期 `IsDefined` 查不到

三、把差异摆成对照表（这一段是关键）

维度	`#if DEBUG` + 特性类	`[Conditional("DEBUG")]` 在特性类上
作用阶段	**预处理**（编译器上班前）	**编译期 emit**（C# 编译器写元数据时）
`TestAttribute` 类本身	DEBUG 关 → **类不存在**	DEBUG 关 → **类还在程序集里**
`[Test]` 使用点合法性	必须也包 `#if`，否则 CS0246	**永远合法**，编译器自己决定写不写元数据
Release 下元数据	特性类 + 使用点都无	特性类有，使用点无
多个条件	`#if DEBUG	
源码整洁度	到处 `#if`，丑	使用点干净，只有定义点一个 `[Conditional]`

四、最扎心的差异：用"反射扫程序集"的第三方视角

假设你写的是**测试框架**，用户用你的 `[Test]`：

`#if` 方案下框架作者的噩梦

用户代码：

```
#if DEBUG
[Test]
#endif
void M() { }
```

用户切 Release 忘了包 `#if` → 编译报错，来骂你。
用户包了 → 你框架 Runner 扫 `Assembly.GetTypes()`，Release 下 `[Test]` 压根没写，OK 是 OK 的，但**用户源码脏。**

`[Conditional]` 方案下

用户代码：

```
[Test]   // 永远合法
void M() { }
```

- DEBUG → Runner 扫到，跑测试 ✅
- Release → Runner 扫不到，不跑 ✅
- 用户**不用改一行**，切配置就行 ✅

五、叠多个 `[Conditional]` 的 OR 语义（顺手提）

```
[Conditional("DEBUG")]
[Conditional("TEST")]
class TestAttribute : Attribute {}
```

```
[Test]
void M() { }
```

→ 只要 `DEBUG` **或** `TEST` 任一定义 → `[Test]` 写进元数据。

这是 **ECMA-335 / C# 规范规定**的 `[Conditional]` 叠多个 = OR。

换成 `#if` 你得写：

```
#if DEBUG || TEST
[Test]
#endif
```

单个还行，多了嵌套就恶心了。

六、还有一个隐藏差异：特性类构造器里的代码

`#if` 方案

```
#if DEBUG
[AttributeUsage(...)]
class TestAttribute : Attribute
{
    public TestAttribute() { Console.WriteLine("hi"); }
}
#endif
```

DEBUG 关 → 构造器**根本不存在**，谈不上执行不执行。

`[Conditional]` 方案

```
[Conditional("DEBUG")]
class TestAttribute : Attribute
{
    public TestAttribute() { Console.WriteLine("hi"); }
}
```

DEBUG 关 → 类还在，**但元数据里没 `[Test]` 这条记录** → 运行期没人 new 它 → 构造器不执行。
（跟 18.6 的"不创建对象"是不同维度：18.6 是运行期查的时候不 new；这里是**元数据里压根没这条配方，运行期连查的机会都没有**）

七、用一句话把这块钉死

> **`#if DEBUG` 是预处理器把"定义特性类 + 使用特性"这两行源码直接删掉，编译器从头到尾不知道它们存在过；**
> **`[Conditional("DEBUG")]` 是特性类源码始终留给编译器，编译器只在 emit 元数据那一刻决定"别人贴的 `[Test]` 要不要写进 CustomAttribute 表"。**
> 前者砍的是文本，后者裁的是元数据。

这也解释了为什么 Richter 说 `[Conditional]` 贴在特性类上是"条件特性类"——**条件作用的是"这个特性被不被记录"，不是"这个类存不存在"。**

这块通了的话，18.7 基本就收了。要不要我把整章（18.1–18.7）用一张"特性生命周期图"收个尾——从你写 `[Xxx]` 到预处理/#if、到编译期 Conditional、到元数据、到 18.4 反序列化、到 18.6 只拿配方——串成一条时间线给你？

（注：文档部分内容可能由 AI 生成）